

Langevin Monte-Carlo

(https://en.wikipedia.org/wiki/Metropolis-adjusted_Langevin_algorithm)

Sampling from an arbitrary differentiable (in $\mathbf{x}$) function eg. Neural network representing $P(\mathbf{x})$ [eg: audio, images]

$$P(\mathbf{x}) \propto \exp(-E_\theta(\mathbf{x})) \quad P(\mathbf{x}) = \frac{\exp(-E_\theta(\mathbf{x}))}{Z_\theta}$$

where $E_\theta(\mathbf{x}) \mapsto R$ is an arbitrary differentiable function in $\mathbf{x}$ and Z_θ is intractable to compute

But given two $\mathbf{x}$ and $\mathbf{x}'$ easy to compute the ratio of their probabilities as $\frac{P(\mathbf{x})}{P(\mathbf{x}')} = \frac{\exp(-E_\theta(\mathbf{x}))}{\exp(-E_\theta(\mathbf{x}'))}$.

How to design $T^Q(\mathbf{x}'|\mathbf{x})$?

Langevin Monte-Carlo: proposal distribution

Intuition: transition from any $\mathbf{x}$ to another $\mathbf{x}'$ along directions of maximum increase in $\log P(\mathbf{x})$ by using gradients.

$$\mathbf{x}' = \mathbf{x} + \tau \nabla_{\mathbf{x}} \log P(\mathbf{x}) = \mathbf{x} - \tau \nabla_{\mathbf{x}} E_{\theta}(\mathbf{x}) + \underbrace{\nabla_{\mathbf{x}} Z_{\theta}}_{=0} \mathbf{x} = 0$$

Making the transitions probabilistic by adding a small Gaussian noise.

$$\mathbf{x}' = \underbrace{\mathbf{x} - \tau \nabla_{\mathbf{x}} E_{\theta}(\mathbf{x})}_{\mu(\mathbf{x}')} + \sqrt{2\tau} \xi, \quad \xi \sim N(0, I_d)$$

$$T^Q(\mathbf{x} \rightarrow \mathbf{x}') = \frac{1}{\sqrt{2\pi}2\tau} \exp\left(-\frac{1}{4\tau} \|\underbrace{\mathbf{x}'}_{P(\mathbf{x}')} - \mathbf{x} + \tau \nabla_{\mathbf{x}} E_{\theta}(\mathbf{x})\|^2\right)$$

$$\underbrace{A(\mathbf{x} \rightarrow \mathbf{x}')}_{P(\mathbf{x})} = \min \left\{ 1, \frac{e^{-E_{\theta}(\mathbf{x}')} T^Q(\mathbf{x}' \rightarrow \mathbf{x})}{e^{-E_{\theta}(\mathbf{x})} T^Q(\mathbf{x} \rightarrow \mathbf{x}')} \right\}$$

Algorithm for Langevin Monte-Carlo

Input: $E_\theta(\mathbf{x})$, Initial sample $\mathbf{x}^1$, τ

1 For $t = 1 \dots M + L$

$$\mathbf{x} = \mathbf{x}^t$$

Compute gradient $\nabla_{\mathbf{x}} E_\theta(\mathbf{x})$

Sample a $\xi \sim N(0, I)$

Compute $\mathbf{x}' = \mathbf{x} - \tau \nabla_{\mathbf{x}} E_\theta(\mathbf{x}) + \sqrt{2\tau} \xi$

Compute $T^Q(\mathbf{x} \rightarrow \mathbf{x}') \propto \exp\left(-\frac{1}{4\tau} \|\mathbf{x}' - \mathbf{x} + \tau \nabla_{\mathbf{x}} E_\theta(\mathbf{x})\|^2\right)$

Compute gradient $\nabla_{\mathbf{x}} E_\theta(\mathbf{x}')$

Compute $T^Q(\mathbf{x}' \rightarrow \mathbf{x}) \propto \exp\left(-\frac{1}{4\tau} \|\mathbf{x} - \mathbf{x}' + \tau \nabla_{\mathbf{x}} E_\theta(\mathbf{x}')\|^2\right)$

$$A = \min \left\{ 1, \frac{e^{-E_\theta(\mathbf{x}')} T^Q(\mathbf{x}' \rightarrow \mathbf{x})}{e^{-E_\theta(\mathbf{x})} T^Q(\mathbf{x} \rightarrow \mathbf{x}')} \right\}$$

Sample a $u \sim U(0, 1)$

$$\mathbf{x}^{t+1} = \mathbf{x}^t \text{ if } u \geq A, \text{ else } \mathbf{x}'$$

2 Return $\mathbf{x}^1, \dots, \mathbf{x}^M$ as samples

Demo

https://colab.research.google.com/drive/1xTrB1DZ2_ZGjq2oFVaJRvsIiY4YPyD5u?usp=sharing

Applications of Langevin Monte Carlo

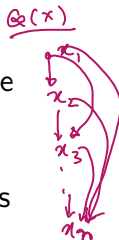
- Protein Discovery with Discrete Walk-Jump Samplingpdf icon
Nathan C. Frey, Dan Berenberg, Karina Zadorozhny, Joseph Kleinhenz, Julien Lafrance-Vanasse, Isidro Hotzel, Yan Wu, Stephen Ra, Richard Bonneau, Kyunghyun Cho, Andreas Loukas, Vladimir Gligorijevic, Saeed Saremi. ICLR 2024.
- Training better generators for high-dimensional objects using deep networks via energy-based networks. SGLB: Stochastic Gradient Langevin Boosting, Aleksei Ustimenko, Liudmila Prokhorenkova, ICML 2021
- Parallel and Flexible Sampling from Autoregressive Models via Langevin Dynamics Vivek Jayaram, John Thickstun, ICML 2021

Sequential Monte Carlo: Motivation

Let $P(\mathbf{x})$ be a target distribution over an n -dimensional variable $\mathbf{x} = x_1, \dots, x_n$.

Assume it is difficult to sample from $P(\mathbf{x}) = \tilde{P}(\mathbf{x}) / Z$

We choose a proposal distribution $Q(\mathbf{x})$ that generates samples auto-regressively. $x_1, x_2 \sim Q(\cdot | x_1), x_3 \sim Q(\cdot | x_1, x_2) \dots$



Earlier we discussed how to use normalized importance sampling (IS) to solve this task

- 1 Get M samples from $Q(\mathbf{x})$

$$x^1, x^2, \dots, x^M \sim Q(\mathbf{x})$$

- 2 Weights: $\tilde{w}(x^i) = \frac{\tilde{P}(x^i)}{Q(x^i)}$ $\tilde{w}_1, \tilde{w}_2, \dots, \tilde{w}_M$

- 3 Resample $c = \text{cat} \left(; \left[\frac{\tilde{w}_1}{\tilde{w}}, \frac{\tilde{w}_2}{\tilde{w}}, \dots, \frac{\tilde{w}_M}{\tilde{w}} \right] \right)$ where $\tilde{w} = \sum_{i=1}^M \tilde{w}_i$

for $i = 1$ to M
 sample an index $w_i \sim \text{cat}(\cdot, [\frac{\tilde{w}_1}{\tilde{w}}, \dots, \frac{\tilde{w}_M}{\tilde{w}}])$
 $\bar{x}^i \leftarrow x^{w_i}$

Return $\bar{x}^1, \dots, \bar{x}^M$

$M = 3$

x^1, x^2, x^3

10 1 3

$\text{cat}(\cdot, [\frac{10}{14}, \frac{1}{14}, \frac{3}{14}]) \leftarrow$

$w^1 \sim 1$

$w^2 \sim 1$

$w^3 \sim 3$

x^1, x^1, x^3

Sequential Monte Carlo: Motivation

- 1 Simple IS is inefficient when most of the weights are small. Several samples needed to obtain good samples from P
- 2 MCMC sampling methods discussed not very useful for models (like LLMs) where variables are sampled sequentially or auto-regressively. $x_i \sim Q(x|x_1, \dots, x_{i-1})$. Gibbs sampling requires conditioning on tokens to the left and to the right.
- 3 In many cases the $P(\mathbf{x})$ can be approximated so that we can get hints of weight even in partial generation. Sequential Monte Carlo methods useful in such settings.

Sequential Monte Carlo: setup

- Let $\underline{P(\mathbf{x})}$ be a target distribution over an n-dimensional variable $\mathbf{x} = x_1, \dots, x_n$.
- We choose a proposal distribution $\underline{Q(\mathbf{x})}$ that generates samples auto-regressively.
-
-
- Design n intermediate distributions $\underline{\pi_1(x_1)}, \dots, \pi_i(\underline{x_1}, \dots, \underline{x_i})$ over prefixes (may be unnormalized). These can be arbitrary as long as $\underline{\pi_n(x_1, \dots, x_n)} = \underline{P(x_1, \dots, x_n)}$. These are meant to be easier subproblems than the final target.

$$\pi_1(x_1) \quad \pi_2(x_1, x_2) \quad , \quad - \quad - \quad - \quad \pi_n(x_1, \dots, x_n) \equiv P(x_1, \dots, x_n)$$

SMC: Main steps I

- 1 Maintain M partial sequences which are grown iteratively from 1 to n by sampling from the proposal distribution $Q(x_t | x_{1:t-1}^k)$.
So, at step t , we will have M partial samples as $x_{1:t}^1, \dots, x_{1:t}^M$

$$\begin{array}{l}
 x_1^1, x_2^1, x_3^1, \dots, x_{t-1}^1 \rightarrow x_t^1 \sim Q(\cdot | x_1^1, x_2^1, \dots, x_{t-1}^1) \\
 x_1^2, x_2^2, x_3^2, \dots, x_{t-1}^2 \rightarrow x_t^2 \\
 \vdots \\
 x_1^M, x_2^M, \dots, x_{t-1}^M \rightarrow x_t^M \sim Q(\cdot | x_1^M, x_2^M, \dots, x_{t-1}^M)
 \end{array}$$

- 2 Define the incremental importance weights

$$w_t^k = \frac{\pi_t(x_{1:t}^k)}{\pi_{t-1}(x_{1:t-1}^k) Q(x_t^k | x_{1:t-1}^k)}$$

SMC: Main steps II

- ③ Resample to clone sequences with high incremental importance weights..Resampling is done with replacement.

$$C \equiv \text{Cat}(\dots [\frac{w_t^1}{w_t}, \frac{w_t^2}{w_t}, \dots, \frac{w_t^M}{w_t}]) \quad w_t = \sum_{i=1}^M w_t^i$$

for $i = 1$ to M {

$$\begin{aligned} w^i &\sim C \\ \bar{x}^i &\sim x^{w^i} \end{aligned}$$

}

SMC Algorithm I

- 1: **Input:** P , proposal Q , intermediate distributions $\{\pi_t\}_{t=1}^n$,
number of particles M
- 2: **for** $t = 1, \dots, n$ **do**
- 3: $\pi_0() = 1$
- 4: **for** $k = 1, \dots, M$ **do**
- 5: Sample $x_t^k \sim Q(x_t \mid x_{1:t-1}^k)$
- 6: Compute incremental weights

$$\underline{w}_t^k = \frac{\pi_t(x_{1:t}^k)}{\pi_{t-1}(x_{1:t-1}^k) Q(x_t^k \mid x_{1:t-1}^k)}$$

- 7: **end for**
- 8: **for** $k = 1, \dots, K$ **do**

SMC Algorithm II

9: Resample ancestor index

$$\omega_t^k \sim \text{Cat} \left(\left\{ \frac{w_t^i}{\sum_{j=1}^K w_t^j} \right\}_{i=1}^K \right)$$

10: Set $\tilde{x}_{1:t}^k$ $\leftarrow x_{1:t}^{\omega_t^k}$

11: **end for**

12: **end for**

13: **Output:** weighted trajectories $\{\tilde{x}_{1:n}^k, \underline{w_n^k}\}_{k=1}^K$

SMC is consistent

IS is consistent

$$x^1, x^2, \dots, x^M \sim q(x)$$

$$\text{Return } (x^1, \frac{\tilde{p}(x^1)}{q(x^1)}), \dots, (x^M, \frac{\tilde{p}(x^M)}{q(x^M)})$$



is consistent:

SMC returns: $\{x^i, w_n^i\}$. We will show that

$$w_n^i = \frac{\tilde{p}(x^i)}{q(x^i)}$$

Proof:

Consider a sample $x_1^i, \dots, x_n^i$

The probability of selection of this sample is

$$\prod_{t=1}^n p(\text{selecting } \underline{x_1^i \dots x_t^i} \text{ via resampling in SMC})$$

$$= \prod_{t=1}^n \frac{\pi_t(x_1^i \dots x_t^i)}{\pi_{t-1}(x_1^i \dots x_{t-1}^i) q(x_t^i | x_1^i \dots x_{t-1}^i)}$$

SMC is consistent

$$\begin{aligned}
 &= \frac{\pi_1(x_1^i)}{\pi_0(x_1)} \cdot \frac{\pi_2(x_1^i, x_2^i)}{\pi_1(x_1^i)} \cdot \dots \cdot \frac{\pi_n(x_1^i, \dots, x_n^i)}{\pi_{n-1}(x_1^i, \dots, x_{n-1}^i)} \frac{\prod_{t=1}^n q(x_t | x_1^i, \dots, x_{t-1}^i)}{1} \\
 &= \frac{\pi_n(x_1^i, \dots, x_n^i)}{\prod_{t=1}^n q(x_t^i | x_1^i, \dots, x_{t-1}^i)} = \frac{\tilde{p}(x)}{q(x)}
 \end{aligned}$$